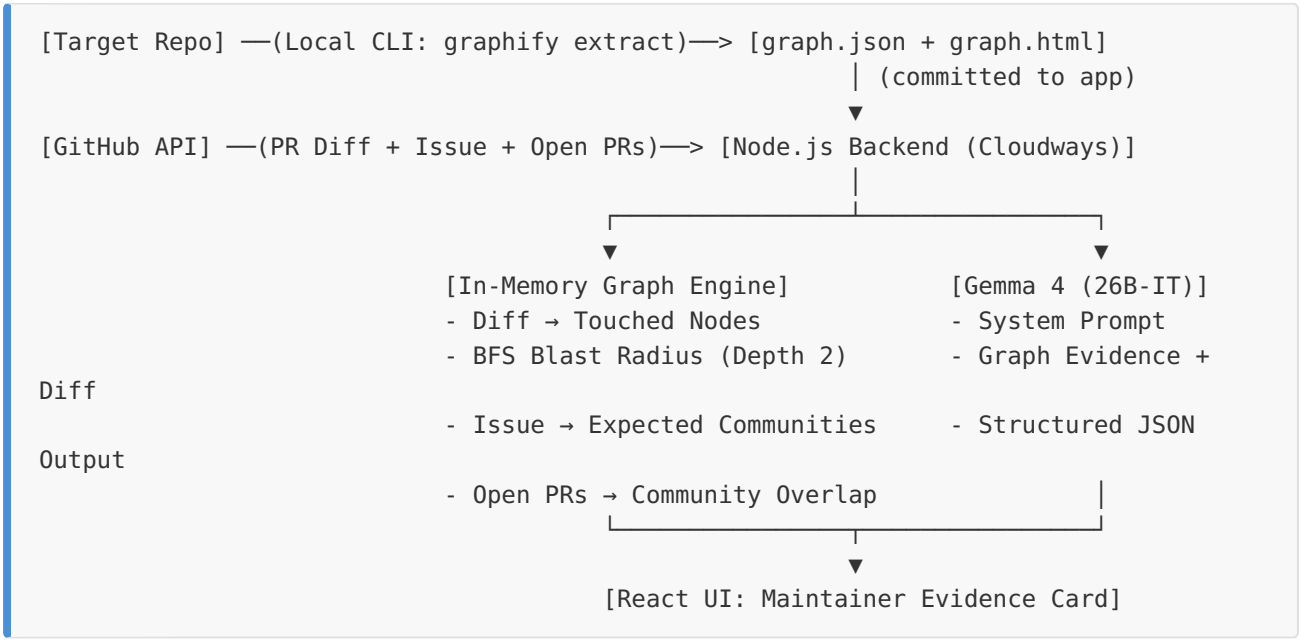
1. High-Level Design & Architecture

PR Reality Check is a single-server Node.js application deployed on Cloudways Velocity. To avoid running Python in production, you pre-generate `graph.json` locally using Graphify (`graphifyy`) against your demo repo and bundle it with the Node app. When a maintainer selects a PR on the React frontend, the Node backend fetches PR metadata, changed files/patches, and linked issue text via the GitHub REST API. An in-memory graph engine maps the changed files/symbols to Graphify nodes, calculates the blast radius (upstream callers + affected tests), and checks community overlap with both the linked issue and other open PRs. Finally, this deterministic graph evidence is passed to **Gemma 4** (`gemma-4-26b-a4b-it`), which synthesizes the structural mismatch into a structured **Evidence Card**.



---

2. The Unique Angle

- **Why Anti-Slop Fails:** Checks account age, PR length, or branch names. A spammer with an old account and a polished LLM-generated description passes easily.
- **Why Generic AI Reviewers Fail:** They only read the `git diff`. If the diff is syntactically clean, the LLM says "LGTM!", completely blind to the fact that it edits the wrong module or breaks 50 downstream callers.
- **Your Winning Edge: Graph-Grounded Verification.** You prove structural reality before calling the LLM.
- **How to Show It in the Demo:** Add a UI toggle on the Evidence Card: **"Standard AI Review (Diff Only)" vs. "Graphify + Gemma 4 Reality Check"**. Show the standard AI praising a clean-looking hallucinated PR, while your tool flags: "0% community overlap with Issue #12; touches God Node *`Router`* with 42 callers."

---

3. Core Algorithm (Step-by-Step)

1. **Map Diff to Graph Nodes (`touchedNodes`):** * Fetch PR files via `GET /repos/:owner/:repo/pulls/:num/files`. * Extract changed file paths and modified function/class names from `@@ ... @@` hunk headers and added/removed lines using regex. * Match against `graph.json` nodes:
   - Primary: Exact file path + line range (if present in `graph.json`).
   - Fallback: Substring match on `node.id` / `node.label` / `node.file` against changed file paths and regex-extracted symbol names.
2. **Compute Blast Radius (Depth Limit = 2):** * Find incoming edges (`target == touchedNode.id`) where label is `EXTRACTED` or `INFERRED`. * Run Breadth-First Search (BFS) up to 2 hops to collect `upstreamCallers`. * Filter `upstreamCallers` for nodes containing `"test"` or `"spec"` to populate `suggestedTests`. * Flag any touched node whose total degree is in the top 5% of the graph as a `godNode`.
3. **Map Linked Issue to Graph (`expectedNodes` & `expectedCommunities`):** * Parse PR body/title for `#<issue_number>` and fetch issue text. * Extract camelCase, snake_case, and `.ext` tokens from the issue title/body. * Match tokens against `graph.json` node labels/paths to find `expectedNodes` and their `community` IDs. * Fallback: If the issue mentions no explicit code names, pass the list of top-level communities (represented by their top 3 node names) to Gemma 4 to infer the expected community.
4. **Claim-vs-Reality Overlap Score:** * Let C_{PR} = set of communities touched by PR diff. * Let C_{Issue} = set of communities mapped from the linked issue. * **Direct Node Hit:** `true` if `touchedNodes` $\cap$ `expectedNodes` $\neq \emptyset$. * **Community Jaccard Score:** $\frac{|C_{\text{PR}} \cap C_{\text{Issue}}|}{|C_{\text{PR}} \cup C_{\text{Issue}}|} \times 100$.
5. **Duplicate / Conflict Detection:** * Fetch open PRs (`GET /repos/:owner/:repo/pulls?state=open`) and their changed files. * Map each open PR to its touched nodes and communities. * If another open PR shares ≥ 1 `touchedNode` or has $> 50\%$ community overlap with the current PR, flag it as a duplicate/conflict.
6. **Gemma 4 Synthesis:** * Pass the calculated graph metrics + PR description + truncated diff (max 4,000 chars) + issue text into a single Gemma 4 prompt. * (Optional Tool Calling): Expose one tool `inspect_graph_node(node_id)` so Gemma can fetch neighbors of an `AMBIGUOUS` edge before finalizing its verdict, or keep it single-shot for speed and reliability.

---

4. MVP Scope vs. Stretch Goals

Must Work by 3:30 PM (MVP)	Cut First If Behind (Stretch Goals)
Pre-generated `graph.json` & `graph.html` for 1 demo repo	Live `graphify` CLI execution on arbitrary repo URLs

Must Work by 3:30 PM (MVP)	Cut First If Behind (Stretch Goals)
Node API reading `graph.json` + fetching live GitHub PRs	GitHub Webhook auto-triggering on PR creation
Graph matcher (Files/Symbols $\rightarrow$ Nodes, Blast Radius, Overlap)	Multi-turn Gemma function calling (single-shot JSON is safer)
Gemma 4 integration returning structured Evidence Card JSON	Posting the Evidence Card back as a GitHub PR comment
Clean React UI showing the 3 demo PRs + served `graph.html`	Ollama local toggle in the UI
Live public URL deployed on Cloudways Velocity + `SKILL.md`	Dynamic repo switcher

---

5. Hour-by-Hour Timeline (4.5 Hours Build Time)

- **10:30 – 11:00 (30m) | Setup, Graph Generation & Cloudways "Hello World"**
- Pick/fork a 20–40 file repo. Run `GRAPHIFY_BACKEND=none graphify extract . --code-only`.
- Inspect `graphify-out/graph.json` to confirm exact keys for `nodes`, `edges`, and `community`.
- Scaffold single Express + Vite/React repo. Deploy "Hello World" to Cloudways Velocity (`CWFEST20`) **immediately** to verify build scripts and env vars.
- **11:00 – 11:45 (45m) | GitHub API & Graph Matching Engine**
- Write `github.js` (fetch PR details, files/patches, issue body, open PR list).
- Write `graphEngine.js` (load `graph.json`, map file/symbol names to nodes, 2-hop BFS blast radius, community overlap, open PR conflict check).
- **11:45 – 12:30 (45m) | Gemma 4 Integration & Cache**
- Connect `@google/genai` with `gemma-4-26b-a4b-it`.
- Feed `graphEngine` output + PR/Issue text into Gemma 4. Parse JSON response and save to a local JSON file cache (`cache.json`).
- **12:30 – 1:30 (60m) | React Dashboard & Evidence Card UI**
- Build the UI: PR selector sidebar, Priority Badge, Claim-vs-Reality comparison box, Blast Radius & God Node warning badges, Duplicate PR alert, and a button to view `graph.html`.
- **1:30 – 2:30 | LUNCH** (Test Cloudways auto-deploy before eating).
- **2:30 – 3:15 (45m) | Demo PR Staging & Value-Adds**
- Open the 3 staged PRs and 2 issues on your forked GitHub demo repo.
- Add the "Standard AI vs. Graphify + Gemma" comparison toggle and commit `SKILL.md`.
- **3:15 – 3:45 (30m) | Production Push & Offline Backup**
- Push final code to Cloudways Velocity. Run all 3 demo PRs on the live URL so they populate the server cache.

- Record a 90-second backup screen capture and write the README.
- **3:45 – 4:00 (15m) | Submission & Pitch Rehearsal**

---

6. Top 6 Judge-Impressing Value-Adds

1. **"Without Graph vs. With Graph" UI Toggle:** Shows side-by-side why diff-only LLM review gets fooled by clean syntax while Graphify + Gemma catches the architectural miss.
2. **Contributor Self-Check Agent Skill (`SKILL.md`):** Include a `SKILL.md` in the repo so coding agents (Claude Code, Cursor, Gemini CLI) can run `graphify query` + Gemma locally before opening a PR. Directly nails the "Best Open-Source AI" category.
3. **Edge Confidence Badges (`EXTRACTED` / `INFERRED` / `AMBIGUOUS`):** Display Graphify's native edge confidence labels on the Blast Radius list so maintainers know whether a dependency is hard AST proof or inferred.
4. **Interactive `graph.html` Embed:** Serve Graphify's generated `graph.html` directly inside an iframe or modal in your React app for instant visual wow-factor.
5. **"Post Evidence to PR" Button:** One-click button that uses your GitHub PAT to post the Markdown version of the Evidence Card directly as a comment on the GitHub PR.
6. **Smart Server Caching with "Force Refresh":** Serves cached results in `<100ms` during the live pitch while allowing a live re-run if judges ask.

---

7. How to Satisfy & Pitch All 4 Prize Categories

Prize Category	How You Satisfy It	Exact 1-Liner for Judges
1. Best Open-Source AI Project	MIT-licensed repo combining open-weight Gemma 4, open-source Graphify, and a reusable `SKILL.md` agent skill to solve the #1 Hacktoberfest 2026 problem.	"Hacktoberfest 2026 stopped counting PRs because of AI spam; we built an open-source maintainer firewall and contributor *`SKILL.md`* powered entirely by open weights."
2. Best Use of Gemma 4	Uses `gemma-4-26b-a4b-it` via `@google/genai` to reason over structured graph topology + code diffs + GitHub issues, plus optional `GRAPHIFY_GEMINI_MODEL` enrichment.	"Gemma 4 doesn't just summarize code diffs—it reasons over Graphify's structural AST topology vs. human intent in the issue tracker."
3. Best Use of Graphify	Uses `graphifyy` communities, node degrees (God nodes), 2-hop blast radius, edge confidence labels (`EXTRACTED` / `INFERRED`), and PR conflict detection.	"Graphify provides the deterministic ground truth—mapping diffs to code communities, God nodes, and cross-PR conflicts before the LLM ever speaks."

Prize Category	How You Satisfy It	Exact 1-Liner for Judges
4. Best Use of Cloudways	Full-stack Node.js app deployed live on Cloudways Velocity via Git with environment variables and live GitHub API integration.	"Our entire triage engine is live right now on Cloudways Velocity—judges can open the public URL and inspect real GitHub PRs in real time."

---

8. Demo Plan & 2-Minute Script

- **Demo Repo to Use:** Fork a clean, recognizable 20–40 file Node/Python project (e.g., a lightweight Express API or FastAPI app with distinct `auth/`, `routes/`, `db/`, and `billing/` modules).
- **The 3 Staged PRs to Create on Your Fork:** 1. **PR #1 (Review First — Genuine Fix):** Links to Issue #10: "Fix token expiration bug in Auth". Modifies `auth/token.js` and `auth/session.js`. High community overlap (100%), low blast radius, suggests `auth.test.js`. 2. **PR #2 (Likely Off-Target — AI Slop):** Links to Issue #11: "Fix login timeout on slow networks". Has a polished AI description claiming it fixes login timeouts, but actually modifies global middleware in `core/router.js` (a **God Node** with 35 callers) and touches 0 files in the `auth` community. 3. **PR #3 (Needs Context — Duplicate/Conflict):** Also attempts to fix Issue #10 by editing `auth/token.js`, directly colliding with PR #1's community and nodes.

Exact 2-Minute Demo Script

- **0:00 – 0:20 (The Hook):** "Hacktoberfest 2026 changed forever because AI-generated spam PRs overwhelmed maintainers. Existing anti-slop bots only check account age or PR size. We built PR Reality Check—hosted live on Cloudways Velocity—which uses Graphify and Gemma 4 to verify if a PR's code changes actually match its claims."
- **0:20 – 1:00 (The Slop Catch - PR #2):** Click PR #2. "Look at PR #2. It claims to fix Issue #11 (login timeout). If I toggle 'Standard AI Review', a normal LLM says 'LGTM, clean code!' Now switch to PR Reality Check: Graphify maps the diff to our repo's knowledge graph and finds 0% overlap with the Auth community. Worse, it modifies *`core/router.js`*, a God Node with 35 downstream callers. Gemma 4 synthesizes this into a red 'Likely Off-Target' card with exact evidence."
- **1:00 – 1:35 (Genuine Fix & Conflict - PR #1 & #3):** Click PR #1. "Now look at PR #1. 100% community match with Issue #10, small blast radius, and Graphify identifies the exact 2 test files to run. Plus, it automatically flags PR #3 as an overlapping duplicate touching the exact same graph community."
- **1:35 – 2:00 (Graph Visual & Open Source Skill):** Click 'View Repo Graph' (*`graph.html`*) & show *`SKILL.md`*. "Maintainers never lose control—we show evidence, never auto-close, and even ship an open-source *`SKILL.md`* so contributors' AI agents can self-check their blast radius before opening a PR."

- **Wi-Fi / API Backup Plan:** Keep `cache.json` committed/saved on the server and in `localStorage` on your laptop, plus a 90-second MP4 screen recording open in a background tab.

---

9. Risks & Quick Fixes

Risk	Quick Fix
`graph.json` lacks line numbers	Match PR changed file paths (`file.filename`) against node IDs/labels, and regex-extract function/class names from git patch headers (`@@ -12,4 +12,8 @@ function foo()`) to fuzzy-match against `node.label` .
Unknown `graph.json` schema	Write a 20-line normalizer right after running `graphify extract` that maps whatever keys exist (`nodes` / `links` or `elements`) into `{ nodes: [{id, label, file, community}], edges: [{source, target, type}] }` .
Gemma 4 Rate Limits (429)	Cache every analyzed PR by `repo + pr_number + head_sha` in a JSON file. Add a deterministic fallback builder that populates the Evidence Card directly from `graphEngine` stats if the Gemini API times out.
Gemma 4 outputs invalid JSON	Strip markdown code fences (```` ```json ... ``` ````) with regex, find the first `{` and last `}` , and wrap `JSON.parse()` in a `try/catch` with a fallback object.
Cloudways Velocity deploy fails	Deploy a single Express app that serves the built React static files (`dist/`) on the same port (`process.env.PORT || 3000`). Zero CORS issues, single build command.
GitHub API 60 req/hr limit	Always pass a `GITHUB_TOKEN` (Personal Access Token) in `Authorization: Bearer` headers to get 5,000 req/hr.

---

10. Sample Artifacts

A. Gemma 4 System Prompt

```
You are PR Reality Check, an architectural verification assistant for open-source
maintainers.
Your job is to compare what a Pull Request CLAIMS to do (PR description + linked
issue) against what it ACTUALLY touches in the codebase knowledge graph (Graphify
evidence).

Rules:
1. Base your technical assessment strictly on the provided GRAPH EVIDENCE and GIT
DIFF. Do not invent file names, callers, or communities.
2. Highlight structural mismatches: e.g., if the issue is about Authentication
(Community A) but the PR only modifies Routing/Config (Community B).
3. Warn the maintainer if the PR touches a high-degree "God Node" or has a large
blast radius.
4. Never recommend auto-closing a PR; provide objective evidence so the human
maintainer can decide.
```

```
5. Respond ONLY with a valid JSON object matching the exact schema requested. Do not
wrap in markdown fences or add conversational text.
```

B. Evidence Card JSON Schema

```
{
  "verdict": "REVIEW_FIRST | NEEDS_CONTEXT | LIKELY_OFF_TARGET",
  "confidenceScore": 92,
  "headline":
"Claims to fix #11 login timeout, but touches 0 Auth nodes and modifies high-risk
Router.",
  "claimVsReality": {
    "claimedIntent": "Fix login timeout on slow connections (Issue #11)",
    "actualChanges": "Modifies global request timeout in core/router.js (Community:
Core Routing)",
    "communityOverlapPct": 0,
    "directSymbolMatch": false,
    "explanation": "Issue #11 points to auth/session.js (Community 2), whereas the
diff only alters core/router.js (Community 0)."
  },
  "blastRadius": {
    "riskLevel": "HIGH",
    "touchedNodes": ["core/router.js::handleRequest"],
    "godNodesTouched": ["core/router.js::handleRequest (35 callers)"],
    "downstreamCallersCount": 35,
    "edgeConfidenceSummary": "31 EXTRACTED, 4 INFERRED",
    "suggestedTests": ["tests/router.spec.js", "tests/api.integration.test.js"]
  },
  "duplicateCheck": {
    "hasOverlap": true,
    "overlappingPRs": [
      { "prNumber": 14, "title": "Refactor router middleware", "sharedCommunities":
["Community 0"] }
    ]
  },
  "maintainerAction": "Ask contributor why core/router.js was modified instead of
auth/session.js before running CI."
}
```

C. Folder Structure (Single-Server Node + React)

```
pr-reality-check/
├── graphify-out/              # Pre-generated locally via graphifyy CLI
│   ├── graph.json             # Loaded by Node backend
│   ├── graph.html             # Served statically for visual modal
│   └── GRAPH_REPORT.md
├── server/
│   ├── index.js               # Express server (API + serves dist/ & graph.html)
│   ├── github.js              # GitHub REST API client (PRs, diffs, issues)
│   ├── graphEngine.js         # Node/Community matcher, BFS blast radius, PR
conflicts
│   ├── gemma.js               # @google/genai (gemma-4-26b-a4b-it) prompt & JSON
parser
│   └── cache.json             # Pre-warmed responses for instant demo fallback
├── client/                    # Vite + React + Tailwind
│   └── src/
```

```
│       ├── App.jsx            # PR list, Diff-Only vs Graph toggle, Evidence Card
│       └── components/
│           └── EvidenceCard.jsx
├── SKILL.md                   # Open-source agent skill for pre-PR self-check
├── package.json
└── README.md
```

D. README Outline

1. **Title & Live Cloudways Demo Link**
2. **The Problem:** Why AI spam broke Hacktoberfest & why surface-level anti-slop filters fail.
3. **How It Works:** Graphify (AST Knowledge Graph) + Gemma 4 (`gemma-4-26b-a4b-it`) = Structural Claim-vs-Reality Verification.
4. **Architecture Diagram** (paste the ASCII diagram from Section 1).
5. **Pre-PR Contributor Agent Skill (`SKILL.md`)** usage instructions.
6. **Quick Start:** `graphify extract`, env vars (`GEMINI_API_KEY`, `GITHUB_TOKEN`), `npm run start`.
7. **Credits & Open-Source Acknowledgments:** `graphifyy`, Google Gemma 4 (`@google/genai`), Cloudways Velocity, Octokit/GitHub REST API, React, Tailwind. (Required by MLH!)

E. Short Submission Pitch (Devpost / MLH)

> **PR Reality Check** protects open-source maintainers from off-target AI pull requests. While existing anti-slop filters only check surface signals like account age or PR size, PR Reality Check uses **Graphify** to map a PR's code changes onto the repository's structural knowledge graph and **Gemma 4** (*`gemma-4-26b-a4b-it`*) to compare those actual graph changes against the PR's stated claims and linked issue. Deployed live on **Cloudways Velocity**, it delivers an instant Maintainer Evidence Card showing Claim-vs-Reality overlap, 2-hop Blast Radius, affected tests, and duplicate open PRs—plus an open-source *`SKILL.md`* so contributors can verify their own PRs before submitting.

First 3 Things to Do Right Now

1. **Generate & Inspect `graph.json` (Next 10 mins):** Fork your demo repo, run `GRAPHIFY_BACKEND=none graphify extract . --code-only`, and open `graphify-out/graph.json` to see the exact JSON property names for nodes, edges, and communities.
2. **Deploy "Hello World" to Cloudways Velocity (Next 15 mins):** Create a minimal Express server with a `/api/health` route, redeem coupon `CWFEST20`, connect your GitHub repo, and verify you have a working public URL before writing complex code.
3. **Test Gemma 4 API Key in Node (Next 5 mins):** Run a 10-line Node script calling `ai.models.generateContent({ model: 'gemma-4-26b-a4b-it', contents: 'Return`

`{"ok":true}' })` to confirm your API key and `@google/genai` SDK work without hiccups.